鸣谢B站UP：我的代码没问题

【蓝桥杯嵌入式省赛快速入门教程（诚心分享+全程干货+免费无套路）】
https://www.bilibili.com/video/BV1uH4y1E7G4?vd_source=172fb8291dc4a2ef1ce47ef40f1be99d

---

/*

- @Author: 洛 (Loren_zzi)
- @Date: 2026-04-02
- @Description:快速入门STM32HAL库&&蓝桥杯嵌入式
  */

代码区域：

因为 CubeMX 是一个图形化配置工具，当你修改了引脚配置并点击“Generate Code”时，它会重新生成 `main.c`。**只有写在 `USER CODE BEGIN` 和 `USER CODE END` 之间的代码会被保留**，写在之外的代码都会被无情地删掉。

```
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
PTD(Private Typedef) -- 私有类型定义
/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
`PD` (Private Define) -- 私有宏定义
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
`PM` (Private Macro) -- 私有宏函数
/* USER CODE END PM */

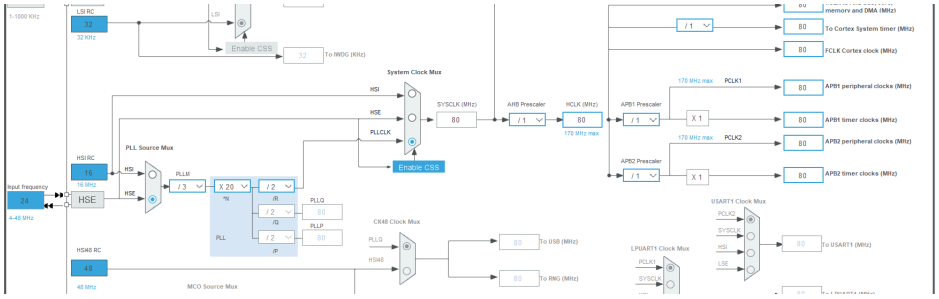
/* Private variables ---------------------------------------------------------*/

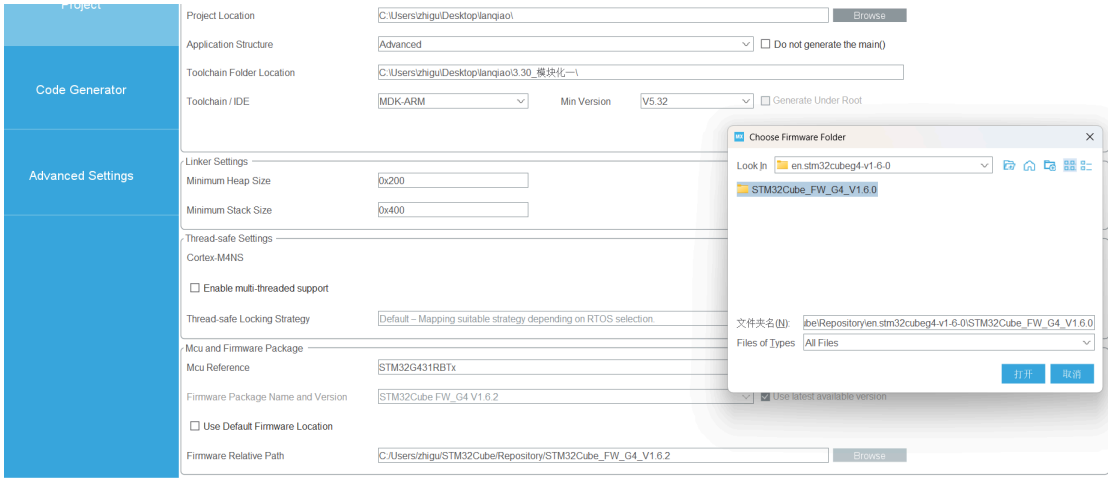
/* USER CODE BEGIN PV */
`PV` (Private Variables) -- 私有变量
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */
(Private Function Prototypes)
/* USER CODE END PFP */
```

点点点：操作

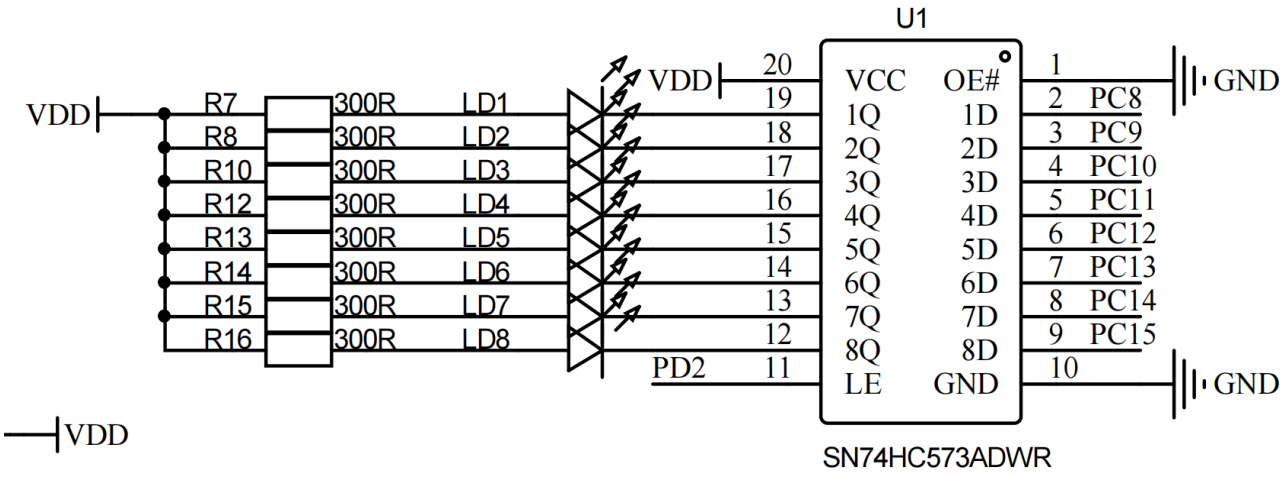
1. （RCC）：外部时钟使能外部高速时钟
2. （SYS）：Debug选择串口
3. 系统时钟配置：



4. 

   并且注意：在Code Generator选择生成.c.h文件
5. 魔术棒 debug选择Reset and Run
6. 增加code分组，也要在魔术棒里添加路径

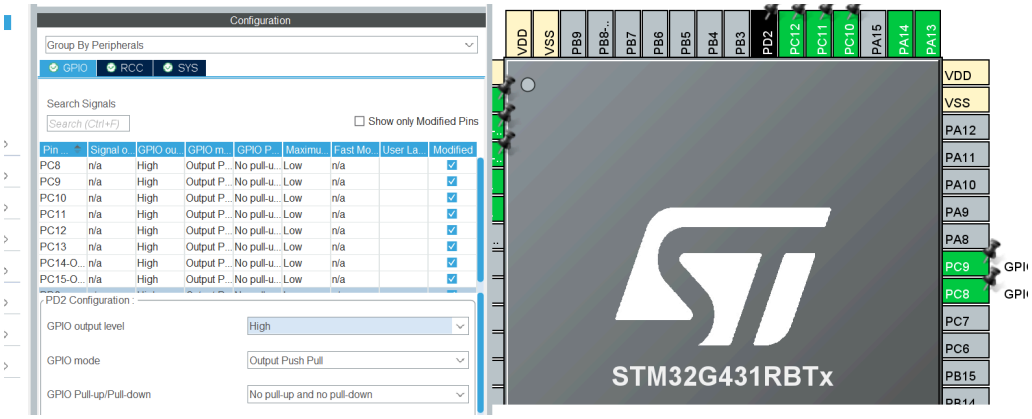
GPIO、LED




```
void show_led(uint8_t led,uint8_t state)
{
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_SET);
    if(state==1)
```

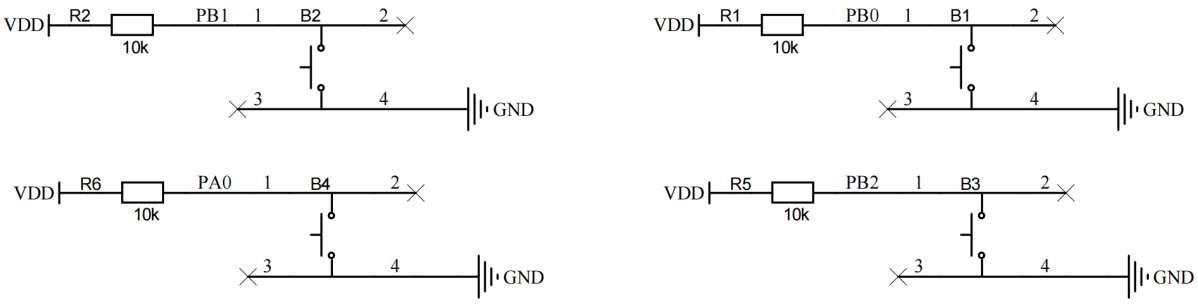
```
    {
        HAL_GPIO_WritePin(GPIOC,0x0080<<led,GPIO_PIN_RESET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOC,0x0080<<led,GPIO_PIN_SET);
    }
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);
}
```

初始化：PC8-PC15 高电平、推挽输出，当译码器打开、IO口为低电平时LED使能。
PD2初始化低电平使能，PD2置高电平时译码器打开（但是为了LED和LCD冲突，要在初始化GPIO时先将PD2为低电平）



PD2 的角色： 它连接到 74HC573 的 **LE (LATCH ENABLE)** 引脚，即“锁存使能端”。
如果不及时拉低 PD2（锁存数据），当调用 LCD 刷新函数时，LCD 的数据信号会传到 GPIOC，由于锁存器还是打开的，你的 **LED 就会跟着 LCD 的数据乱闪。*！！！所以每次操作完LED就要立即将PD2拉低！***

按键：



初始化：PB0、1、2、PA0为上拉输入模式
当HAL_GPIO_ReadPin读到为0时，说明按键按下

Pin ...	Signal o...	GPIO ou...	GPIO m...	GPIO P...	Maximu...	Fast Mo...	User La...	Modified
PA0	n/a	n/a	Input mo...	Pull-up	n/a	n/a		☑
PB0	n/a	n/a	Input mo...	Pull-up	n/a	n/a		☑
PB1	n/a	n/a	Input mo...	Pull-up	n/a	n/a		☑
PB2	n/a	n/a	Input mo...	Pull-up	n/a	n/a		☑
PC8	n/a	High	Output P...	No pull-u...	Low	n/a		☑
PC9	n/a	High	Output P...	No pull-u...	Low	n/a		☑

PA0#PB0#PB1#PB2 Configuration :

GPIO mode: Input mode

GPIO Pull-up/Pull-down: Pull-up

下面介绍一种非阻塞式的写法：

状态机逻辑本质是检测按下是否达到长按键阈值、否则是短按键

在中断和主循环中共同访问的变量，必须加 `volatile` 防止被编译器优化掉

``

```
//////////////////////////////////////在key.c中
//============================key
// 定义全局变量时加上 volatile
//在中断和主循环中共同访问的变量，必须加 `volatile` 防止被编译器优化掉
volatile uint8_t key_short;
volatile uint8_t key_long;

static uint8_t key_time=0;
static uint8_t long_state=1;

uint8_t key_state(void)
{
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_0)==0) return 1;
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_1)==0) return 2;
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_2)==0) return 3;
    if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==0) return 4;

    return 0;
}

void key_loop(void)
{
    static uint8_t last_state=0,now_state=0;

    last_state=now_state;
    now_state=key_state();

    uint8_t i;
    for(i=1;i<=4;i++)
    {
        if(last_state==0&&now_state==i)
        {
            key_time=0;
            long_state=1;    //重置长按标志位
        }
```

```
        if(last_state==i&&now_state==i)
        {
            if(++key_time==40&&long_state==1)
            {
                key_long=i;
                long_state=0;
            }
        }

        if(last_state==i&&now_state==0)
        {
            if(long_state==1)
            {
                key_short=i;
            }
            else
            {
                long_state=1;
            }
        }
    }


}
//////////////////////////////在定时器中每20ms刷新一次函数key_loop
uint8_t led7_state;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    static uint16_t count_1s,count_20ms;
    if(htim->Instance==TIM3)  //10ms
    {
        if(++count_1s==100)
        {
            count_1s=0;
            led7_state++;
            led7_state=led7_state%2;
        }
        if(++count_20ms==2)
        {
            count_20ms=0;
            key_loop();
        }

    }

}
//////////////////////////////////////在主函数while里调用:
      if(key_short)
      {
        switch(key_short)
        {
            case 1:
            {
```

```
                count1++;
            }break;

            case 3:
            {
                led2_state=(++led2_state)%2;
                show_led(2,led2_state);
            }break;

            case 4:
            {
                count2++;
            }break;

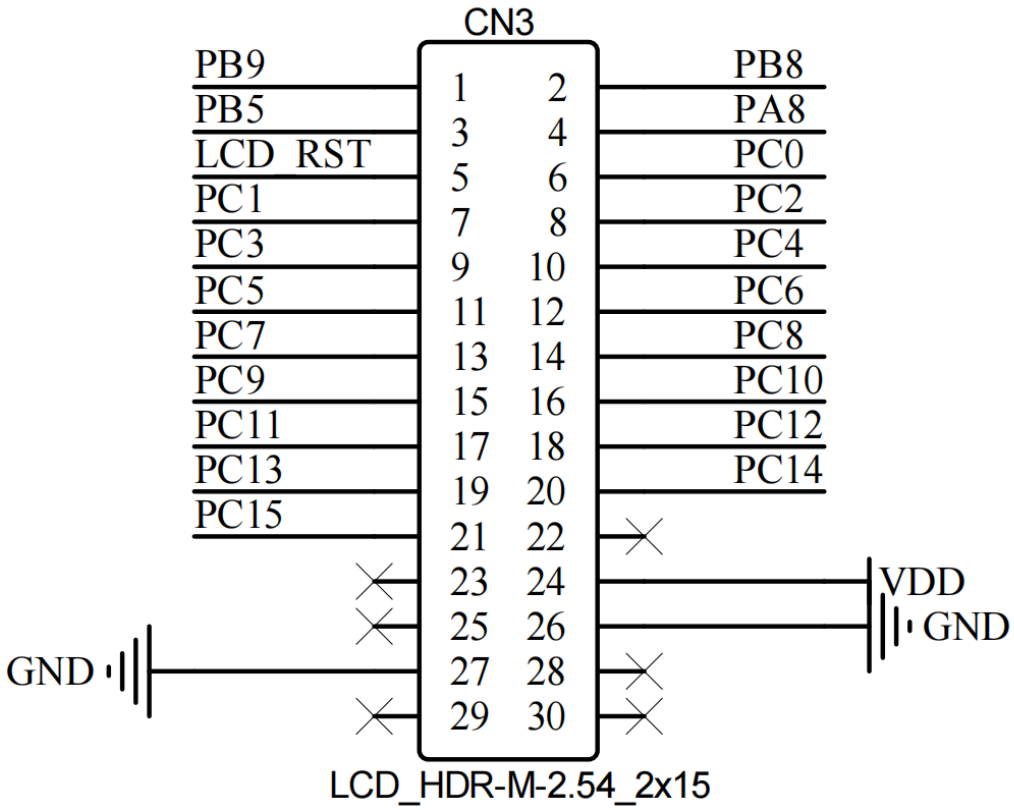
      }key_short=0;
    }

        if(key_long)
    {
      switch(key_long)
      {
          case 1:
          {
              count1--;
          }break;

          case 4:
          {
              count2--;
          }break;

      }key_long=0;
    }
```

LCD：



初始化：

将Src中lcd.c、Inc中fonts.h好有lcd.h文件复制到我们工程code里面

在官方给的LCD.c文件里已经帮你初始化过，无需再手动配置

```
  LCD_Init();
  LCD_Clear(Black);
  LCD_SetBackColor(Black);
  LCD_SetTextColor(White);
```

每一行最多20个字符

```
char text[20];
void show_lcd(void)
{
    sprintf(text,"       text:");
    LCD_DisplayStringLine(Line0,(u8 *)text);
    sprintf(text,"    count:  %-3d",count);
    LCD_DisplayStringLine(Line1,(u8 *)text);
}
```

关于**残余字符** 问题：使用%3d或者%-3d，使得无需显示的用空格占替。

为什么一个text变量就能显示、这是人眼视觉残留效应吗？

答：不是，是利用了 LCD 硬件自带的“显存”功能

某一行高亮：

`LCD_SetBackColor` 确实是设置一个全局变量（通常叫 `BackColor`），但它**不会立刻把已经画在屏幕上的东西变色**。它影响的是**在这行代码之后**执行的“打印”动作。

```
void show_lcd(void)
{
    // 第一步：Line0
    LCD_SetBackColor(highlight_line == 0 ? Yellow : Black);
    sprintf(text, "   hello,world!   ");
    LCD_DisplayStringLine(Line0, (u8*)text);

    // 第二步：Line1
    LCD_SetBackColor(highlight_line == 1 ? Yellow : Black);
    sprintf(text, "    count1:  %-3d", count1);
    LCD_DisplayStringLine(Line1, (u8*)text);

    // 第三步：Line2
    LCD_SetBackColor(highlight_line == 2 ? Yellow : Black);
    sprintf(text, "    count2:  %-3d", count2);
    LCD_DisplayStringLine(Line2, (u8*)text);

    // 最后记得切回黑色，防止影响其他地方
    LCD_SetBackColor(Black);
}
```

关于LED和LCD共用引脚冲突问题：

1. 初始化LED时将PD2置高电平、后手动拉低PD2低电平

```
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */
  HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);
```

2. 在每一个关于LCD的函数内加上代码：

```
u16 temp = GPIOC->ODR; // 进门前：把 GPIOC 现在的状态（包含 LED 的亮灭情况）拍个照存下来
// ... 中间任由 LCD 驱动怎么折腾 GPIOC 都没关系 ...
GPIOC->ODR = temp;     // 出门前：按照刚才拍的照片，把 GPIOC 恢复原样

例如初始化：
void LCD_Init(void)
{
    u16 temp=GPIOC->ODR;
    LCD_CtrlLinesConfig();
    dummy = LCD_ReadReg(0);

    if(dummy == 0x8230)
    {
        REG_8230_Init();
    }
```

```
    else
    {
        REG_932X_Init();
    }
    dummy = LCD_ReadReg(0);
    GPIOC->ODR=temp;
}
```

3. 再加上你在调用 LCD 前，通过 `HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);` 把锁存器关死了。这样 LCD 刷屏幕的时候，数据进不到 LED 里；等屏幕刷完，你又把 `GPIOC` 恢复到了刷屏幕前的状态。这样 LED 和 LCD 就实现了完美的**分时复用**。

定时器的一些参数及其关系（主频、PSC、ARR、CCR）

要把定时器（Timer）玩透，理解这四个参数之间的“级联”关系是基本功。我们可以把定时器想象成一个**“往木桶里自动注水”**的过程。

1. 核心成员介绍

在 STM32 中，定时器的核心是由一个 **16位/32位计数器（CNT）** 驱动的。

参数名称	全称	形象比喻	作用
f_{sys}	**主频 (System Clock)**	水流的速度	定时器的原始能量来源。
PSC	**预分频器 (Prescaler)**	水龙头开关（减速器）	决定计数器“跳动”一次需要多少时间。
ARR	**自动重装载寄存器 (Auto-Reload)**	木桶的高度	决定计数器数到多少就“溢出”重启，即**周期**。
CCR	**捕获/比较寄存器 (Capture/Compare)**	水桶里的浮标（阈值）	决定 PWM 的**占空比**或记录**捕获时刻**。

2. 它们之间的级联逻辑

第一步：分频 ($f_{sys} \rightarrow PSC$)

原始的主频太快了（比如 80MHz），直接计数会导致计数器瞬间爆表。

- **频率计算**：计数器频率 $f_{CNT} = \frac{f_{sys}}{PSC+1}$。
- **物理意义**：这就是你之前问的 $80-1$ 的由来。设为 79，则 $f_{CNT} = 1\text{MHz}$，意味着计数器每 $1\mu s$ 加 1。

第二步：决定周期 ($PSC \rightarrow ARR$)

计数器从 0 开始往上数，数到 ARR 的值时，就会清零重新开始。

- **溢出频率**：$f_{overflow} = \frac{f_{CNT}}{ARR+1} = \frac{f_{sys}}{(PSC+1)(ARR+1)}$。
- **应用**：这就是你输出 PWM 波形的频率，或者是定时器中断（如你的 `htim3`）触发的频率。

第三步：决定占空比 ($ARR \rightarrow CCR$)

*这是在 **PWM 模式**下最常用的逻辑。计数器 CNT 在从 0 到 ARR 增长的过程中，会不断和 CCR 比较。*

- **逻辑**：当 $CNT < CCR$ 时，引脚输出高电平；当 $CNT \geq CCR$ 时，引脚输出低电平（假设是极性向上）。
- **占空比 (Duty Cycle)**：$Duty = \frac{CCR}{ARR+1}$。

$$f_{PWM} = \frac{f_{sys}}{(PSC+1) \times (ARR+1)}$$

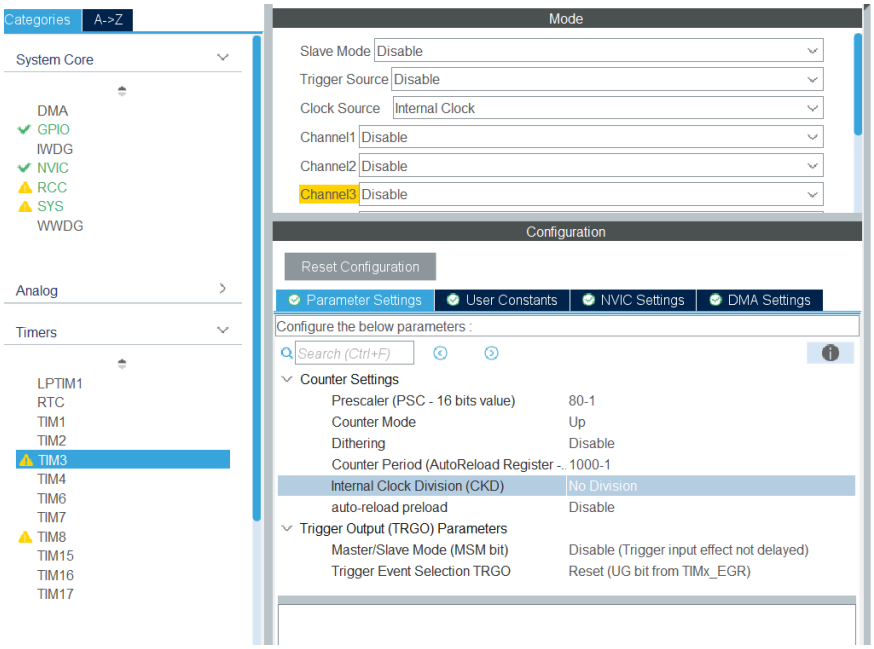
$$Duty = \frac{CCR}{ARR+1} \times 100\%$$

定时器：

$$f = 80000000/(psc+1)/(arr+1)$$

所以周期就是：
$T = \frac{(psc+1)\times(arr+1)}{80\,000\,000}$

一般配置10ms就够用了：
10ms psc=80-1 arr=10000-1
1ms: psc=80-1 arr=1000-1
这是1ms的，记住打开内部时钟，还有中断使能



初始化使能

```
HAL_TIM_Base_Start_IT(&htim3);
```

使用回调函数：

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
```

```
    static uint16_t count_1s,count_20ms;
    if(htim->Instance==TIM3)  //10ms
    {
        if(++count_1s==100)
        {
            count_1s=0;
            led7_state++;
            led7_state=led7_state%2;
        }
        if(++count_20ms==2)
        {
            count_20ms=0;
            key_loop();
        }

    }

}
```

PWM：

频率：

$$f = \frac{f_{system}}{(ARR+1)(PSC+1)}$$

CCR - 输出比较寄存器

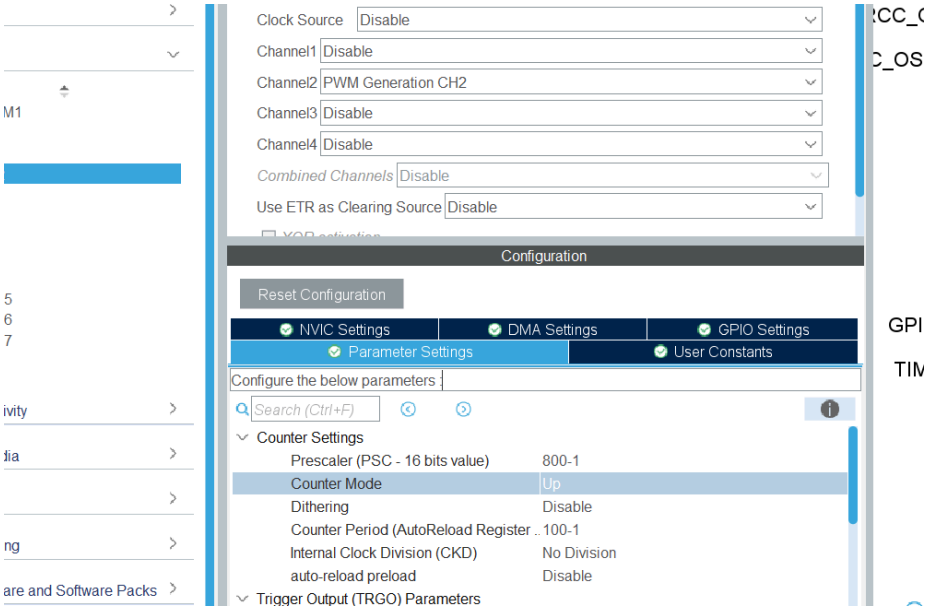
周期：

$$T = \frac{1}{f} = \frac{(ARR+1)(PSC+1)}{f_{system}}$$

占空比：

$$\frac{CCR}{ARR+1}$$

如果：让PA1输出频率为1000Hz，占空比为50%的方波
那么在将PA1配置成定时器如TIM2_CH2,PSC=800-1,ARR=100-1，这样输出频率就是1000HZ。



main初始化中：设置占空比为百分之五十

```
HAL_TIM_PWM_Start(&htim2,TIM_CHANNEL_2);
TIM2->CCR2=50;
```

IC（输入捕获）：

```
$f_{CNT} = \frac{f_{sys}}{PSC + 1}$
```

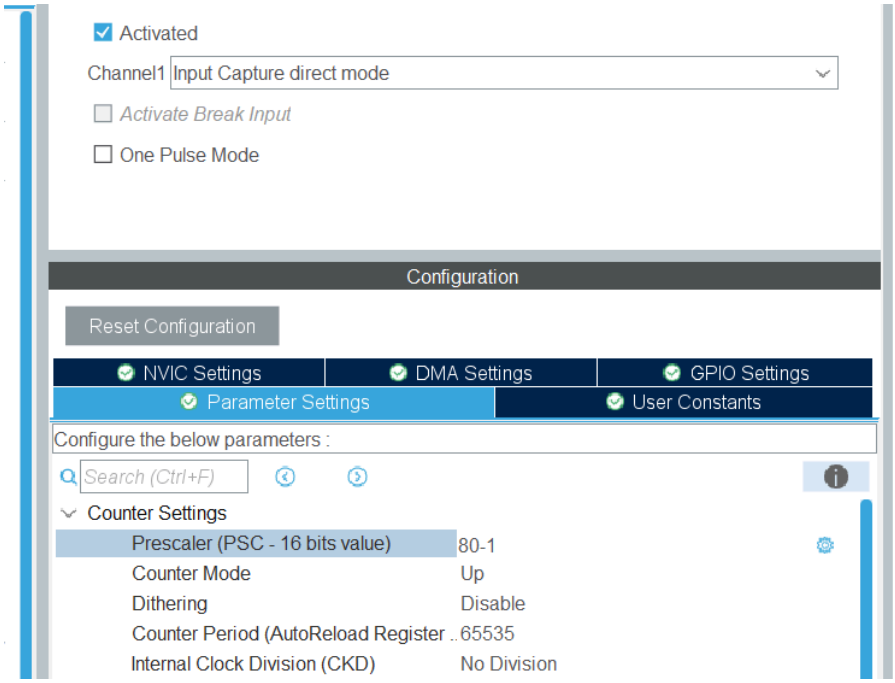
$$T = t_0 \cdot \text{capture_value}$$

$$f = \frac{1}{T} = \frac{f_{sys}}{(PSC+1) \cdot \text{capture_value}}$$

输入捕获引脚PWM频率：

利用PA7输入捕获上面的PA1输出的PWM：

1. cubemx配置PA7为TIM17定时器，记得要中断使能
   只关心PSC就可以，因为需要的是CNT的周期



2，IC中断使能启动：

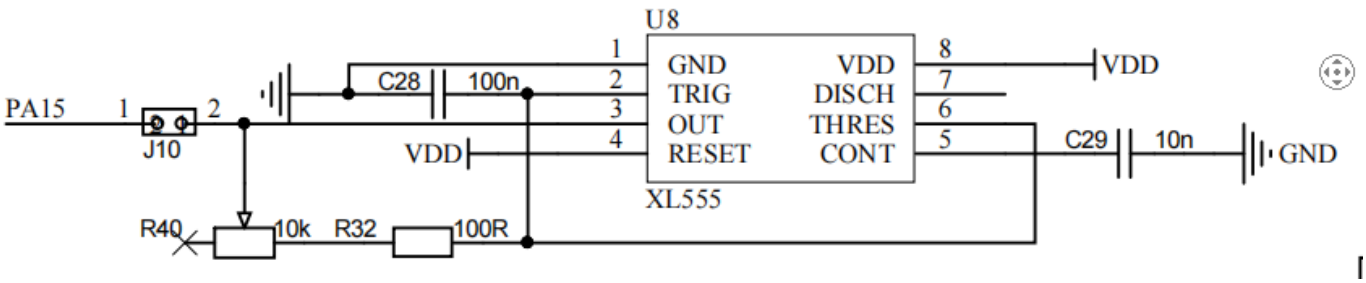
```
HAL_TIM_IC_Start_IT(&htim17,TIM_CHANNEL_1);
```

3，中断读取CCR赋值给capture，并计算

当外部信号的上升沿触发时，硬件会自动把那一瞬间计数器 `CNT` 的数值“锁存”到 `CCR1` 中

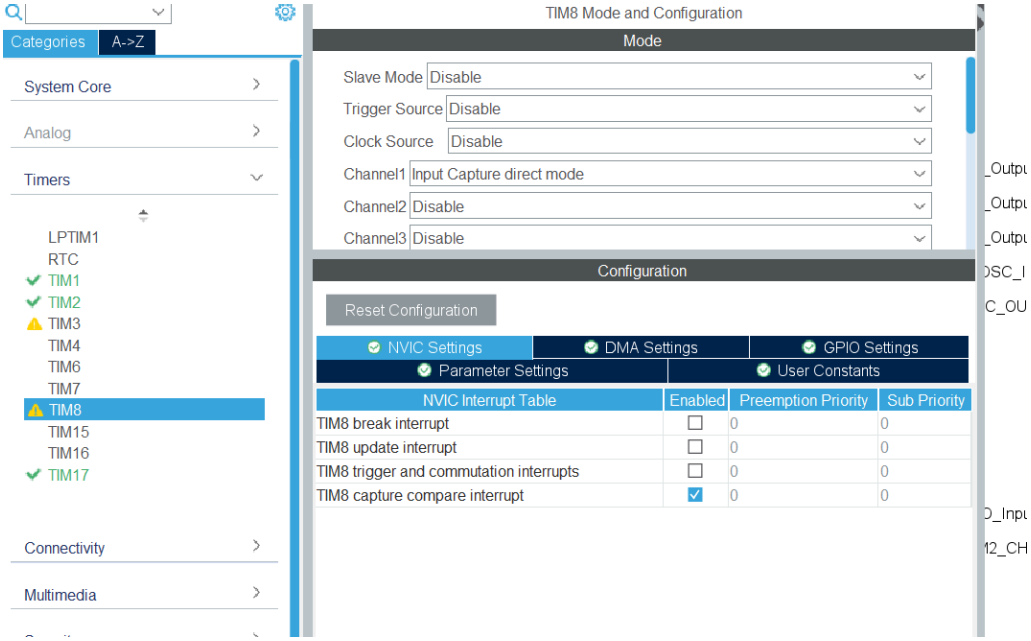
```
uint32_t fre,capture_val;
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance==TIM17)
    {
        capture_val=TIM17->CCR1;
        //capture_val=HAL_TIM_ReadCapturedValue(htim,TIM_CHANNEL_1);
        TIM17->CNT=0;
        fre=80000000/(80*capture_val);
    }
}
```

输入捕获555定时器产生的PWM波形



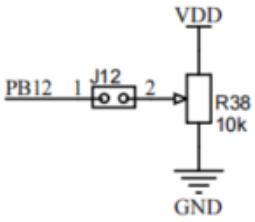
555定时器通过R40变阻器产生不同频率的PWM，可用内部的PA15引脚来输入捕获

原理和上面一样

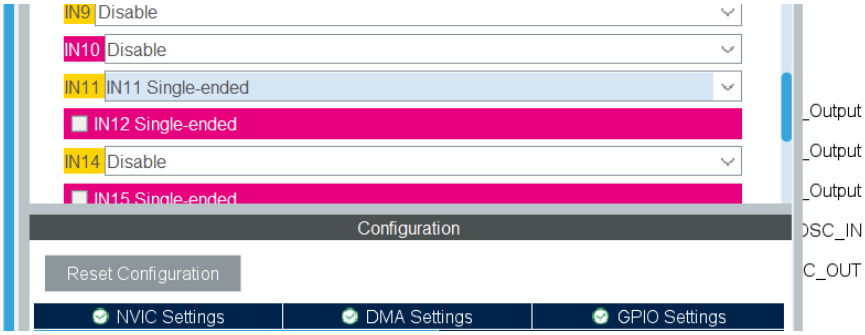



```
HAL_TIM_IC_Start_IT(&htim8,TIM_CHANNEL_1);
    if(htim->Instance==TIM8)
    {
        //capture_val=HAL_TIM_ReadCapturedValue(htim,TIM_CHANNEL_1);
        capture_val2=TIM8->CCR1;
        TIM8->CNT=0;
        Fre_NE555=80000000/(80*capture_val2);
    }
```

ADC测量：



将PB2配置为ADC1_IN11,选择为IN11 Single-ended模式（输入电压对地比较，另一种模式是differential差分模式：输入与输入比较）



在cubemx配置完后之后会生成一个adc文件，编译一下，将这个adc.h文件放在我们headfile.h里面

在LCD里面启动、获取、刷新：

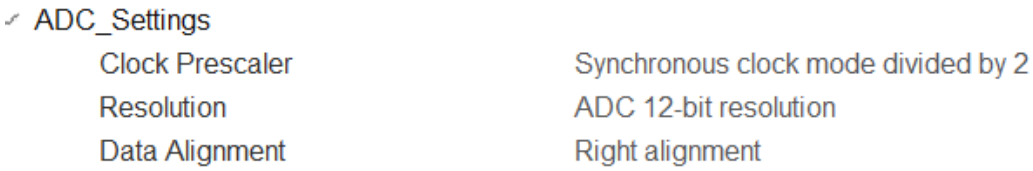
```
    HAL_ADC_Start(&hadc1);
    adc_val= HAL_ADC_GetValue(&hadc1);

        sprintf(text,"    adc_val:%5d    ",adc_val);
        LCD_DisplayStringLine(Line8,(u8*)text);
```

获取电压：

```
double get_vol(ADC_HandleTypeDef *hadc)
{
    HAL_ADC_Start(hadc);
    adc_val= HAL_ADC_GetValue(hadc);
    return 3.3*adc_val/4096;
}
//然后在LCD中直接显示
        sprintf(text,"    adc_val:%5.2f    ",get_vol(&hadc1));
        LCD_DisplayStringLine(Line8,(u8*)text);
```

为什么是除以4096?



因为这里ADC设置为12位量化

STM32 HAL库核心概念：句柄与指针规则

1. 命名缩写解析

- **前缀 `h`**：代表 `Handle`（句柄）。
  - `htim` = Handle of Timer（定时器句柄）
  - `hadc` = Handle of ADC（ADC句柄）
  - `huart` = Handle of UART（串口句柄）

2. 什么是“句柄 (Handle)”?

- **代码本质**：句柄是一个**结构体变量**（例如 `TIM_HandleTypeDef htim2;`），内部包含了该外设的：
  1. 物理寄存器基地址指针（`.Instance`）
  2. 初始化配置参数（`.Init`）
  3. 当前工作状态（Ready / Busy / Error 等）
- **作用**：HAL库函数通过接收不同外设的句柄，实现对不同硬件的统一化操作管理。

3. 句柄 (`htim2`) vs 底层寄存器 (`TIM2`)

名称格式	例子	属性	作用与用法
小写带 `h`	`htim2`	**软件结构体变量**	用于所有 `HAL_...` 开头的库函数传参。 例: `HAL_TIM_PWM_Start(&htim2, ...);`

名称格式	例子	属性	作用与用法
全大写无 `h`	`TIM2`	**物理寄存器基地址**	用于直接操作底层寄存器，执行速度极快。例：`TIM2->CCR2 = 50;`
关联关系	`htim2.Instance == TIM2`		

4. `&` (取地址符) 的使用铁律

HAL 库的操作函数形参通常要求传入**结构体的指针**。是否加 `&` 取决于当前变量的类型：

情况 A：必须加 `&` (取地址)

- **场景**：变量是全局定义的**结构体实体**（如在 main.c 中定义的 `TIM_HandleTypeDef htim2;`）。
- **原因**：函数需要指针，而你手里是实体，必须用 `&` 取出它的内存地址。
- **示例**：

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
HAL_ADC_Start(&hadc1);
```

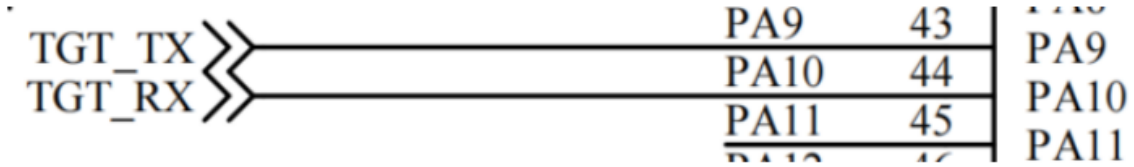
情况 B：坚决不能加 `&`

- **场景**：变量本身**已经是一个指针**（常见于回调函数或自定义函数的形参）。
- **原因**：它已经是地址了，再取地址就变成了“指针的指针”，会导致程序直接崩溃（HardFault）。
- **示例**：

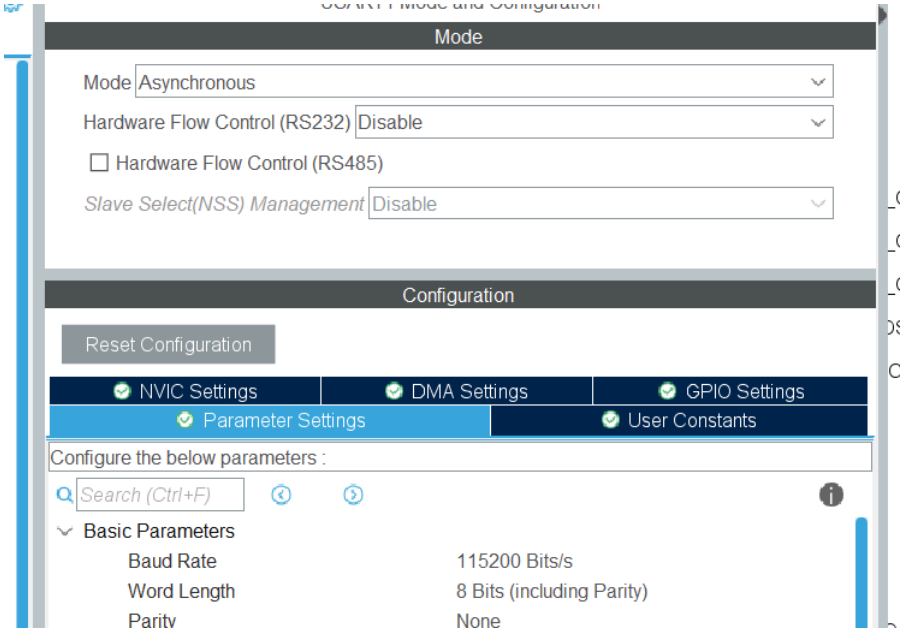
```
// 形参 htim 带有 *，说明它已经是指针
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    // 1. 调用HAL函数直接传指针变量名
    HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

    // 2. 访问内部成员必须用 -> 符号
    if(htim->Instance == TIM8)
    { ... }
}
```

串口：



将PA10配置为UART_TX,PA11为UART_RX，Mode选择Asynchronous异步通信，记得要使能中断



发送字符串：

其中HAL_UART_Transmit函数第三个参数要用关键字strlen：只计算字符串的实际长度（sizeof直接用数组大小，可能会有被污染的内存发送）

```
      char text01[20];
      sprintf(text01,"hello world\r\n");
      HAL_UART_Transmit(&huart1,(uint8_t *)text01,strlen(text01),50);
      HAL_Delay(1000);//会阻塞
```

接收一个字符：

```
//////////初始化中：
  HAL_UART_Receive_IT(&huart1,&rec_date,1);//相当于串口使能
  //////////中断中
  uint8_t  rec_date;
  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance==USART1)
    {
        HAL_UART_Transmit(huart,&rec_date,1,50);//最后的50是超时机制，最多等50ms就会跳出去
        HAL_UART_Receive_IT(huart,&rec_date,1);
    }

}
```

sprintf到底是什么

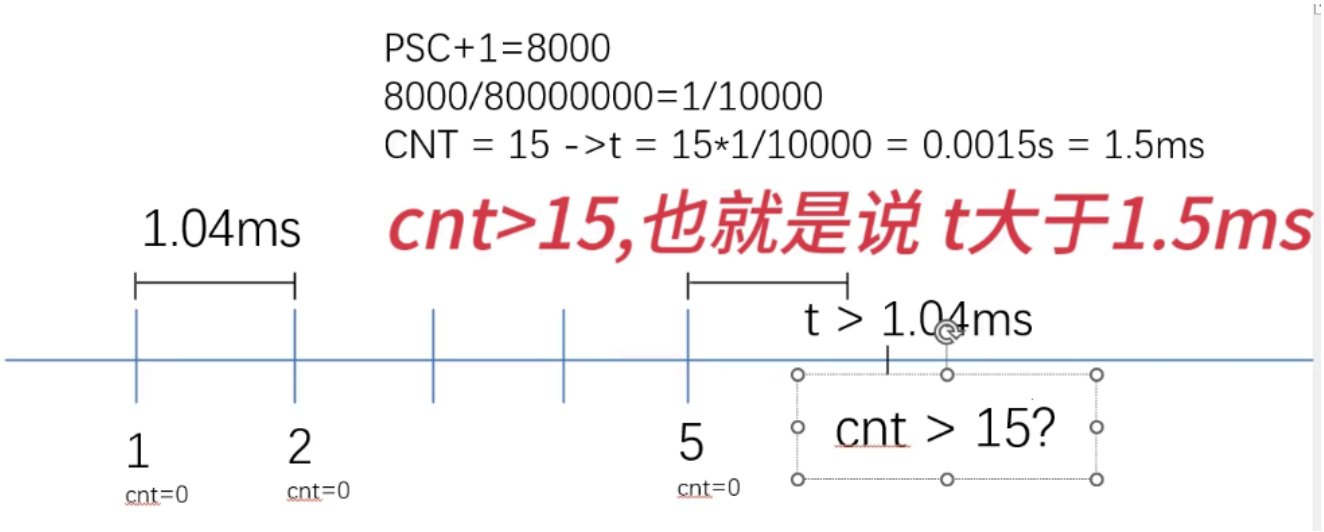
全称是 **string print formatted** 核心作用就是将这些数值**格式化**成我们可以阅读的**字符串**

回调函数：为什么要反复调用 `HAL_UART_Receive_IT`？

你在 `HAL_UART_RxCpltCallback` 中重复调用：`HAL_UART_Receive_IT(huart, &rec_date, 1);`

- **单次触发属性**：HAL 库的中断接收是“一次性”的。当它收完你指定的长度（比如 1 个字节）并触发回调函数后，硬件中断就会自动关闭。
- **接力赛机制**：为了能持续接收后续数据，你必须在处理完当前字节后，立刻“重新布防”，告诉硬件：“请继续盯着，收完下一个再叫我”。

使用定时器接收字符串：



思路：字符串里的字符都是连着发的，波特率是9600的话那么每一个字符（10bit）大约花费1.04s，那么我们只需要判断间隔是否大于1.04ms就可以了，这时候我们就可以操作定时器，每次开始接收到一个字节cnt都清零，如果哪次大于阈值了还没有清零就说明后面没有字符了也就表明字符串结束了

```
uint8_t rec_data,count0;
uint8_t rec_flag;
uint8_t rec_buff[20];
char send_buff[20];

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance==USART1)
    {
        TIM4->CNT=0;
        //HAL_UART_Transmit(huart,&rec_data,1,50);
        rec_flag=1;
        rec_buff[count0++]=rec_data;
        HAL_UART_Receive_IT(huart,&rec_data,1);
    }
}

void uart_date_rec(void)
{
    if(rec_flag)
    {
        if(TIM4->CNT>=20)
        {
            if(rec_buff[0]=='a'&&rec_buff[1]=='b')
```

```
            {
                sprintf(send_buff,"abc");
                HAL_UART_Transmit(&huart1,(uint8_t *)send_buff,strlen(send_buff),50);
            }
            else
            {
                sprintf(send_buff,"error");
                HAL_UART_Transmit(&huart1,(uint8_t *)send_buff,strlen(send_buff),50);
            }
              // --- 核心清零操作 ---
            // 重置状态
            rec_flag=0;
            count0=0;
            memset(rec_buff, 0, sizeof(rec_buff));

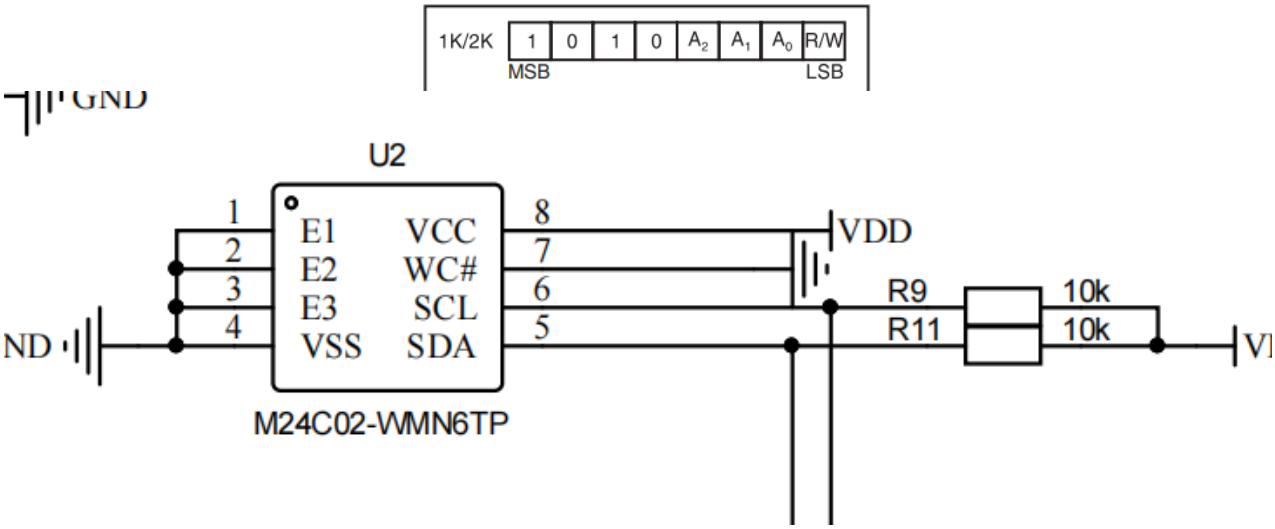
        }
    }
}
```

这里有一个注意点！：在主循环中刷新LCD太快会影响uart_date_rec函数判断导致失效！
除了这种定时器超时法，实际工程中更常使用的是 **串口空闲中断 (IDLE) + DMA**

---

eeprom—AT24C02

Figure 7. Device Address



所以地址就是：0101000+{读写位} 读是1，0是写 也就是0xa0/0xa1；

```
//1是读取 0是写
void eeprom_write(uint8_t addr,uint8_t dat)
{
    I2CStart();
    I2CSendByte(0xa0);
    I2CWaitAck();
    I2CSendByte(addr);
    I2CWaitAck();
```

```
    I2CSendByte(dat);
    I2CWaitAck();
    I2CStop();

    HAL_Delay(20);
}


uint8_t eeprom_read(uint8_t addr)
{
//第一步将AT24C02内部指针对准要读取的位置
    I2CStart();
    I2CSendByte(0xa0);
    I2CWaitAck();
    I2CSendByte(addr);
    I2CWaitAck();
    I2CStop();

//正式开始读数据
    I2CStart();
    I2CSendByte(0xa1);
    I2CWaitAck();
    uint8_t dat=I2CReceiveByte();
    I2CSendNotAck();
    I2CStop();

    return dat;
}
```

在初始化中：

```
I2CInit();
eeprom_write(0,10);
uint8_t dat=eeprom_read(0);
```

RTC实时时钟

比较简单这里就水了，当达到闹钟时led8就会亮

```
char text[20];
  RTC_TimeTypeDef sTime = {0};
  RTC_DateTypeDef sDate = {0};
  uint8_t led8state;
void show_lcd(void)
{
      HAL_RTC_GetTime(&hrtc,&sTime,RTC_FORMAT_BIN);
      HAL_RTC_GetDate(&hrtc,&sDate,RTC_FORMAT_BIN);
    sprintf(text,"  Hello,World!");
```

```
    LCD_DisplayStringLine(Line0,(uint8_t *)text);
    sprintf(text,"  %2d:%2d:%2d    ",sTime.Hours,sTime.Minutes,sTime.Seconds);
    LCD_DisplayStringLine(Line1,(uint8_t *)text);
    sprintf(text,"  %d-%d-%d      ",sDate.Year,sDate.Month,sDate.WeekDay);
    LCD_DisplayStringLine(Line2,(uint8_t *)text);

    show_led(8,led8state);
}
void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc)
{
    led8state=1;
}
```

完结撒花！！！ ❀❀❀